JOUR 04 BIS / TP 03 — V2 · VERSION GUIDÉE (V2 bis)

Establish Ground Link

Remplacer les mocks par `GET` véhicules et missions, derrière des services typés, avec un `useEffect` correctement cadré.

---

*Durée indicative : **40 à 45 min** · Organisation : **individuel** · Évaluation :** non noté*

Support : IDE, à partir du TP 2 · JSON `public/api/` du starter, ou API de séance

Contexte

Une réponse HTTP n'est pas encore une donnée fiable. Les interfaces TypeScript **disparaissent** à l'exécution : `response.json() as Vehicle[]` documente une attente, elle ne valide rien.

Trois catégories de logique, pas une :

- **rendu** : calculer ce qui s'affiche (filtre, total) — **pas** un effet ;
- **événement** : clic / submit — **pas** un effet qui surveille un booléen `shouldSend` ;
- **effet** : synchroniser React avec un système externe (API, timer). C'est le cas ici.

Un effet décrit une synchronisation : connecter, lire, **nettoyer**. Les dépendances ne sont pas un menu. Le classique qui boucle :

TypeScript · À éviter — boucle infinie

```
useEffect(() => {
  fetchVehicles().then(setVehicles)
}, [vehicles])
```

L'effet écrit `vehicles`, qui relance l'effet. Sans `AbortController`, une requête hors-écran peut encore `setState` après démontage. Une erreur d'annulation n'est **pas** une panne sol.

Séparez : type DTO reçu → vérification minimale → transformation vers le modèle UI → affichage. Les services reçoivent un `AbortSignal`. Ils lèvent si `!response.ok`. Ils ne rendent pas de JSX.

Résultat attendu

1. `FleetPage` n'importe plus le tableau mock comme source principale : la liste vient d'un `GET` (`/api/vehicles.json` ou `VITE_API_URL/vehicles`).

2. `MissionsPage` idem pour les missions.

3. `src/services/vehicles.ts` et `src/services/missions.ts` existent. Chaque service accepte `signal?: AbortSignal`, teste `response.ok`, parse en `unknown` (ou via un helper `getJson`).

4. Un `useEffect` charge au montage : dépendances **stables** (pas le tableau résultat), cleanup `controller.abort()`, ignore `AbortError`.

5. Mini-défi : `{ "vehicle_id": 7, "fuel_ratio": 0.82, "state": "GO" }` devient `{ id: "7", fuelPercent: 82, status: "ready" }` dans le modèle UI (alignez les champs sur **votre** type).

6. Un HTTP non-2xx n'est pas une liste vide (console ou écran). Le rendu complet loading / empty / error est le TP 4.

7. `npx tsc --noEmit` est vert.

Étapes

1. Brancher l'URL du sol

Le starter sert souvent ces fichiers JSON. Un `fetch` vers le chemin public suffit.

Chemin · public/api

```
public/api/vehicles.json
public/api/missions.json
```

TypeScript · fetch local

```
fetch("/api/vehicles.json")
```

Si le formateur donne une API : `VITE_API_URL` dans `.env.local`, documentée dans `.env.example` **sans secret**. Si la clé manque, levez une erreur claire, pas un `fetch("undefined/vehicles")`.

Chemin · Variables d'environnement

```
.env.local          → VITE_API_URL=…
.env.example        → VITE_API_URL=   (sans secret)
```

2. Services

Même schéma pour les missions.

Chemin · Fichiers à créer

```
src/services/vehicles.ts
src/services/missions.ts
```

TypeScript · src/services/vehicles.ts

```
export async function getVehicles(signal?: AbortSignal) {
  const response = await fetch("/api/vehicles.json", { signal })
  if (!response.ok) throw new Error(`HTTP ${response.status}`)
  const data: unknown = await response.json()
  return data
}
```

Pas de `as Vehicle[]` sur le JSON brut. Une garde (`Array.isArray`, champs présents) ou un helper `getJson` avec un commentaire « l'assertion ne valide pas » est acceptable.

DTO optionnel : ces types décrivent le **fil**, pas l'UI.

Chemin · src/types/dto.ts

```
vehicle_id
fuel_ratio
state
```

3. Adaptateur — ne pas contaminer l'UI

Si les JSON du starter sont **déjà** au format `LaunchVehicle`, documentez-le en une ligne dans le service. Le mini-défi reste obligatoire : créez l'adaptateur et testez-le à la main (`console.log` ou petit assert).

Chemin · src/services/adaptVehicle.ts

```
src/services/adaptVehicle.ts
```

JSON · Mini-défi — entrée sol

```
{ "vehicle_id": 7, "fuel_ratio": 0.82, "state": "GO" }
```

TypeScript · Mini-défi — modèle UI

```
{ id: "7", fuelPercent: 82, status: "ready" }
```

- `vehicle_id` → `id: string`
- `fuel_ratio` 0–1 → `fuelPercent` 0–100 entier
- `state: "GO"` → `status: "ready"`

Aucun `fuel_ratio` dans un composant.

4. `useEffect` dans les pages — le bon, pas la boucle

Dans `FleetPage` et `MissionsPage` :

Chemin · Pages à brancher

```
src/pages/FleetPage.tsx
src/pages/MissionsPage.tsx
```

- `useState` pour le résultat (ou une union provisoire) ;
- `useEffect` qui crée un `AbortController`, appelle le service, `setState` en succès ;
- dépendances : `[]` pour une liste globale, **jamais** `[vehicles]` / `[missions]` ;
- `return () => controller.abort()` ;
- `AbortError` : `return` silencieux, pas un message d'erreur.

TypeScript · useEffect — dépendances stables + abort

```ts
useEffect(() => {
  const controller = new AbortController()
  async function load() {
    try {
      const data = await getVehicles(controller.signal)
      // adapter puis setState
    } catch (error) {
      if (error instanceof DOMException && error.name === "AbortError") return
      console.error(error)
    }
  }
  void load()
  return () => controller.abort()
}, [])
```

Interdit : filtrer la flotte « ready » dans un second effet. Calculez au rendu :

TypeScript · Rendu — pas un effet

```ts
const ready = vehicles.filter((v) => v.status === "ready")
```

Les cartes, badges et `Link` du TP 2 doivent continuer à marcher. Ne fusionnez pas mock local + API.

5. Vérifier

- `/fleet` et `/missions` : une requête dans l'onglet Réseau (deux en Strict Mode au plus, **pas** une boucle).
- Changer de route pendant le fetch : pas de warning `Can't perform a React state update on a component that unmounted`.
- Couper le JSON ou forcer une 404 : une erreur visible (console ou écran), pas une liste inventée.
- Contrôler l'adaptateur du mini-défi.

Terminal · Terminal

```
npx tsc --noEmit
```

Livrables

Chemin · Fichiers

```
src/services/vehicles.ts
src/services/missions.ts
src/services/adaptVehicle.ts
```

- pages branchées, effet avec abort, pas de boucle
- type-check sans erreur
- commit :

Terminal · Git

```
git add src
git commit -m "feat(session-04): establish ground link services"
```

Suite — TP 4 Signal Lost

Le lien sol existe, mais l'écran ne raconte encore que le succès. Au TP 4 vous distinguerez **idle, loading, success, empty, error** : skeleton, HTTP 500, tableau vide, retry, et Hermes sans carburant — sans jamais inventer `0 %`.